

Jenkins: De Principiante a Experto

Guía Completa de CI/CD con Ejemplos Prácticos

Apuntes de Desarrollo

26 de noviembre de 2025

Índice general

Prefacio

Jenkins es una de las herramientas más populares para Integración Continua y Despliegue Continuo (CI/CD). Este documento te llevará desde los conceptos más básicos hasta implementaciones avanzadas, con ejemplos prácticos que podrás usar en proyectos reales.

Capítulo 1

Introducción a Jenkins

1.1. ¿Qué es Jenkins?

Jenkins es una herramienta de automatización de código abierto escrita en Java que facilita la integración continua y el despliegue continuo (CI/CD) de proyectos de software. Fue creada por Kohsuke Kawaguchi en 2004 y se ha convertido en la herramienta estándar de la industria para automatización de builds.

Características Principales

- **Open Source:** Completamente gratuito y con código abierto
- **Multiplataforma:** Funciona en Windows, macOS, Linux
- **Extensible:** Más de 1800 plugins disponibles
- **Distribuido:** Soporta arquitecturas master-slave
- **Pipeline como código:** Jenkinsfile para definir procesos

1.2. Conceptos Fundamentales

1.2.1. Integración Continua (CI)

La Integración Continua es una práctica de desarrollo donde los desarrolladores integran código en un repositorio compartido frecuentemente, idealmente varias veces al día. Cada integración se verifica mediante builds automáticos.

1.2.2. Despliegue Continuo (CD)

El Despliegue Continuo extiende la CI automatizando el proceso de liberación de software, permitiendo que cada cambio que pase las pruebas se despliegue automáticamente en producción.

1.2.3. Jobs/Trabajos

Un job en Jenkins es una tarea configurable que ejecuta una serie de pasos. Puede ser:

- **Freestyle:** Configuración básica con interfaz gráfica
- **Pipeline:** Definido como código (Jenkinsfile)
- **Multi-branch:** Automáticamente crea pipelines para ramas
- **Organization folder:** Escanea organizaciones completas

1.3. Arquitectura de Jenkins

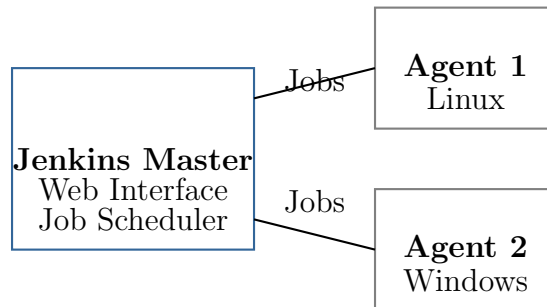


Figura 1.1: Arquitectura Master-Agent de Jenkins

Capítulo 2

Instalación y Configuración

2.1. Instalación en Linux

2.1.1. Mediante Docker (Recomendado)

```
1 # Crear volumen para persistencia
2 docker volume create jenkins_home
3
4 # Ejecutar Jenkins
5 docker run -d \
6   --name jenkins \
7   -p 8080:8080 \
8   -p 50000:50000 \
9   -v jenkins_home:/var/jenkins_home \
10  -v /var/run/docker.sock:/var/run/docker.sock \
11  jenkins/jenkins:lts
12
13 # Ver logs para obtener la contraseña inicial
14 docker logs jenkins
```

Listing 2.1: Instalación con Docker

2.1.2. Instalación Nativa en Ubuntu

```
1 # Actualizar repositorios
2 sudo apt update
3
4 # Instalar Java (prerequisito)
5 sudo apt install openjdk-11-jdk -y
6
7 # Agregar clave GPG de Jenkins
8 wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo
   apt-key add -
9
10 # Agregar repositorio
11 sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ > /etc
   /apt/sources.list.d/jenkins.list'
12
13 # Instalar Jenkins
14 sudo apt update
15 sudo apt install jenkins -y
```

```

16
17 # Iniciar y habilitar servicio
18 sudo systemctl start jenkins
19 sudo systemctl enable jenkins
20
21 # Ver estado
22 sudo systemctl status jenkins

```

Listing 2.2: Instalación en Ubuntu

2.2. Configuración Inicial

2.2.1. Primer Acceso

Después de la instalación, accede a <http://localhost:8080>:

1. Obtener contraseña inicial:

```

1 sudo cat /var/lib/jenkins/secrets/initialAdminPassword
2

```

2. Seleccionar "Install suggested plugins"
3. Crear usuario administrador
4. Configurar URL de Jenkins

2.2.2. Configuración de Seguridad

Mejores Prácticas de Seguridad

- Habilitar autenticación basada en matriz
- Configurar usuarios y roles específicos
- Usar HTTPS en producción
- Configurar firewall apropiadamente
- Actualizar Jenkins regularmente

Capítulo 3

Primeros Pasos con Jenkins

3.1. Creando tu Primer Job

3.1.1. Job Freestyle

1. Navegar a "New Item"
2. Seleccionar "Freestyle project"
3. Configurar:
 - **Source Code Management:** Git repository
 - **Build Triggers:** Poll SCM cada 5 minutos
 - **Build Steps:** Execute shell command

```
1 #!/bin/bash
2 echo "Iniciando build..."
3 echo "Fecha: $(date)"
4 echo "Usuario: $(whoami)"
5 echo "Directorio: $(pwd)"
6
7 # Compilar proyecto (ejemplo)
8 if [ -f "package.json" ]; then
9     npm install
10    npm test
11 elif [ -f "pom.xml" ]; then
12     mvn clean test
13 elif [ -f "requirements.txt" ]; then
14     pip install -r requirements.txt
15     python -m pytest
16 fi
17
18 echo "Build completado exitosamente!"
```

Listing 3.1: Ejemplo de Build Step

3.2. Configuración de Git

3.2.1. Credenciales

1. Ir a "Manage Jenkins" → "Manage Credentials"

2. Agregar credenciales de tipo "Username with password"
3. Para repositorios privados, usar SSH keys o tokens

```
1 # Generar clave SSH
2 ssh-keygen -t rsa -b 4096 -C "jenkins@yourcompany.com"
3
4 # Agregar clave pública al repositorio Git
5 cat ~/.ssh/id_rsa.pub
6
7 # En Jenkins, usar "SSH Username with private key"
```

Listing 3.2: Configurar SSH Key

Capítulo 4

Jenkins Pipelines

4.1. Introducción a Pipelines

Los pipelines son la forma moderna de definir trabajos en Jenkins. Se escriben como código (Jenkinsfile) y ofrecen mayor flexibilidad y control.

4.1.1. Pipeline Declarativo vs Scripted

Pipeline Declarativo (Recomendado)

Sintaxis más simple y estructurada, más fácil de leer y mantener.

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Checkout') {
6       steps {
7         git branch: 'main',
8           url: 'https://github.com/usuario/proyecto.git'
9       }
10    }
11
12    stage('Build') {
13      steps {
14        sh 'echo "Building application..."'
15        sh 'npm install'
16      }
17    }
18
19    stage('Test') {
20      steps {
21        sh 'npm test'
22      }
23    }
24
25    stage('Deploy') {
26      steps {
27        sh 'echo "Deploying application..."'
28      }
29    }
30  }
```

```
30 }
31
32 post {
33     success {
34         echo 'Pipeline completed successfully!'
35     }
36     failure {
37         echo 'Pipeline failed!'
38     }
39 }
40 }
```

Listing 4.1: Pipeline Declarativo Básico

4.2. Elementos del Pipeline

4.2.1. Agent

Define dónde ejecutar el pipeline:

```
1 pipeline {
2     // Cualquier agente disponible
3     agent any
4
5     // Agente específico por etiqueta
6     agent { label 'linux' }
7
8     // Usando Docker
9     agent {
10         docker {
11             image 'node:14'
12         }
13     }
14
15     // Sin agente (definir en cada stage)
16     agent none
17 }
```

Listing 4.2: Tipos de Agent

4.2.2. Stages y Steps

```
1 pipeline {
2     agent any
3
4     environment {
5         NODE_VERSION = '14'
6         APP_NAME = 'mi-aplicacion'
7     }
8
9     stages {
10         stage('Checkout') {
11             steps {
12                 checkout scm
13                 script {
14                     env.GIT_COMMIT_SHORT = sh(
```

```

15         script: 'git rev-parse --short HEAD',
16         returnStdout: true
17     ).trim()
18     env.BUILD_VERSION = "${BUILD_NUMBER}-${env.
GIT_COMMIT_SHORT}"
19     }
20 }
21 }
22
23 stage('Compile') {
24     steps {
25         sh 'mvn clean compile -DskipTests=true'
26     }
27 }
28
29 stage('Unit Tests') {
30     steps {
31         sh 'mvn test -Dspring.profiles.active=test'
32     }
33     post {
34         always {
35             publishTestResults testResultsPattern: 'target/
surefire-reports/*.xml'
36             publishCoverage adapters: [
37                 jacocoAdapter('target/site/jacoco/jacoco.xml')
38             ], sourceFileResolver: sourceFiles('STORE_LAST_BUILD
')
39         }
40     }
41 }
42
43 stage('Code Quality Analysis') {
44     parallel {
45         stage('Checkstyle') {
46             steps {
47                 sh 'mvn checkstyle:check'
48             }
49             post {
50                 always {
51                     recordIssues enabledForFailure: true, tools:
[checkStyle()]
52                 }
53             }
54         }
55
56         stage('SpotBugs') {
57             steps {
58                 sh 'mvn spotbugs:check'
59             }
60             post {
61                 always {
62                     recordIssues enabledForFailure: true, tools:
[spotBugs()]
63                 }
64             }
65         }
66     }
67 }

```

```
68
69     stage('Integration Tests') {
70         steps {
71             sh 'mvn verify -Dspring.profiles.active=test -
DskipUnitTests=true'
72         }
73         post {
74             always {
75                 publishTestResults testResultsPattern: 'target/
failsafe-reports/*.xml'
76             }
77         }
78     }
79
80     stage('Package') {
81         steps {
82             sh "mvn package -DskipTests=true -Dspring.profiles.
active=${SPRING_PROFILES_ACTIVE}"
83             archiveArtifacts artifacts: 'target/*.jar',
allowEmptyArchive: false
84         }
85     }
86
87     stage('Security Scan') {
88         steps {
89             sh 'npm audit --audit-level high'
90         }
91     }
92
93     stage('Deploy to Staging') {
94         when {
95             branch 'develop'
96         }
97         steps {
98             sh 'echo "Deploying to staging..."'
99             sh './deploy-staging.sh'
100         }
101     }
102
103     stage('Deploy to Production') {
104         when {
105             branch 'main'
106         }
107         steps {
108             input message: 'Deploy to production?', ok: 'Deploy'
109             sh 'echo "Deploying to production..."'
110             sh './deploy-production.sh'
111         }
112     }
113 }
114
115 post {
116     always {
117         cleanWs()
118     }
119     success {
120         slackSend channel: '#deployments',
121             color: 'good',
```



```
122         message: "      ${env.JOB_NAME} - ${env.BUILD_NUMBER}"
123     deployed successfully"
124     }
125     failure {
126         slackSend channel: '#deployments',
127                 color: 'danger',
128                 message: "      ${env.JOB_NAME} - ${env.BUILD_NUMBER}"
129     failed"
130     }
131 }
```

Listing 4.3: Pipeline Completo con Múltiples Stages

Capítulo 5

Jenkins para Proyectos Java

5.1. Herramientas Específicas para Java

5.1.1. Configuración de Maven

Maven es la herramienta de build más utilizada en el ecosistema Java:

```
1 # Descargar y configurar Maven
2 wget https://dlcdn.apache.org/maven/maven-3/3.8.6/binaries/apache-maven
   -3.8.6-bin.tar.gz
3 tar xzf apache-maven-3.8.6-bin.tar.gz
4 sudo mv apache-maven-3.8.6 /opt/maven
5
6 # Configurar en Jenkins: Manage Jenkins      Global Tool Configuration
7 # Maven installations:
8 # Name: Maven-3.8.6
9 # MAVEN_HOME: /opt/maven
```

Listing 5.1: Instalación de Maven en Jenkins

5.1.2. Comandos Maven Esenciales

Comandos Maven para CI/CD

- **mvn clean:** Limpia archivos generados
- **mvn compile:** Compila código fuente
- **mvn test:** Ejecuta tests unitarios
- **mvn verify:** Ejecuta tests de integración
- **mvn package:** Genera JAR/WAR
- **mvn install:** Instala en repositorio local
- **mvn deploy:** Despliega a repositorio remoto

```
1 <build>
2   <plugins>
3     <!-- Surefire para tests unitarios -->
```

```

4      <plugin>
5          <groupId>org.apache.maven.plugins</groupId>
6          <artifactId>maven-surefire-plugin</artifactId>
7          <version>3.0.0-M7</version>
8          <configuration>
9              <includes>
10                 <include>**/*Test.java</include>
11                 <include>**/*Tests.java</include>
12             </includes>
13         </configuration>
14     </plugin>
15
16     <!-- Failsafe para tests de integraci n -->
17     <plugin>
18         <groupId>org.apache.maven.plugins</groupId>
19         <artifactId>maven-failsafe-plugin</artifactId>
20         <version>3.0.0-M7</version>
21         <configuration>
22             <includes>
23                 <include>**/*IT.java</include>
24                 <include>**/*IntegrationTest.java</include>
25             </includes>
26         </configuration>
27     </plugin>
28
29     <!-- JaCoCo para cobertura de c digo -->
30     <plugin>
31         <groupId>org.jacoco</groupId>
32         <artifactId>jacoco-maven-plugin</artifactId>
33         <version>0.8.7</version>
34         <executions>
35             <execution>
36                 <goals>
37                     <goal>prepare-agent</goal>
38                 </goals>
39             </execution>
40             <execution>
41                 <id>report</id>
42                 <phase>test</phase>
43                 <goals>
44                     <goal>report</goal>
45                 </goals>
46             </execution>
47         </executions>
48     </plugin>
49 </plugins>
50 </build>

```

Listing 5.2: Configuraci3n de Plugins Maven para CI/CD

5.2. An3lisis de C3digo en Java

5.2.1. Checkstyle

Checkstyle verifica que el c3digo siga est3ndares de codificaci3n:

```

1 <plugin>

```

```

2    <groupId>org.apache.maven.plugins</groupId>
3    <artifactId>maven-checkstyle-plugin</artifactId>
4    <version>3.1.2</version>
5    <configuration>
6        <configLocation>google_checks.xml</configLocation>
7        <encoding>UTF-8</encoding>
8        <consoleOutput>true</consoleOutput>
9        <failsOnError>true</failsOnError>
10    </configuration>
11 </plugin>

```

Listing 5.3: Configuración de Checkstyle

```

1 stage('Checkstyle') {
2     steps {
3         sh 'mvn checkstyle:check'
4     }
5     post {
6         always {
7             recordIssues enabledForFailure: true, tools: [checkStyle()]
8         }
9     }
10 }

```

Listing 5.4: Integración en Pipeline

5.2.2. SpotBugs

SpotBugs encuentra bugs potenciales en el código Java:

```

1 <plugin>
2     <groupId>com.github.spotbugs</groupId>
3     <artifactId>spotbugs-maven-plugin</artifactId>
4     <version>4.7.1.1</version>
5     <configuration>
6         <effort>Max</effort>
7         <threshold>Low</threshold>
8         <xmlOutput>true</xmlOutput>
9     </configuration>
10 </plugin>

```

Listing 5.5: Configuración de SpotBugs

5.2.3. OWASP Dependency Check

Verifica vulnerabilidades de seguridad en dependencias:

```

1 <plugin>
2     <groupId>org.owasp</groupId>
3     <artifactId>dependency-check-maven</artifactId>
4     <version>7.1.1</version>
5     <configuration>
6         <format>ALL</format>
7         <suppressionFile>owasp-suppressions.xml</suppressionFile>
8     </configuration>
9 </plugin>

```

Listing 5.6: OWASP Dependency Check

5.3. Testing Avanzado en Java

5.3.1. Tests Unitarios con JUnit 5

```

1 @ExtendWith(MockitoExtension.class)
2 class UsuarioServiceTest {
3
4     @Mock
5     private UsuarioRepository usuarioRepository;
6
7     @InjectMocks
8     private UsuarioService usuarioService;
9
10    @BeforeEach
11    void setUp() {
12        // Configuración inicial para cada test
13    }
14
15    @Test
16    @DisplayName("Deber a crear usuario cuando email no existe")
17    void crearUsuario_EmailNoExiste_DeberiaCrearExitosamente() {
18        // Given
19        Usuario usuario = new Usuario("Test", "test@example.com", "123")
20    ;
21        when(usuarioRepository.existsByEmail(anyString())).thenReturn(
22        false);
23        when(usuarioRepository.save(any())).thenReturn(usuario);
24
25        // When
26        Usuario resultado = usuarioService.crearUsuario(usuario);
27
28        // Then
29        assertAll(
30            () -> assertNotNull(resultado),
31            () -> assertEquals("Test", resultado.getNombre()),
32            () -> assertEquals("test@example.com", resultado.getEmail())
33        );
34
35        verify(usuarioRepository).existsByEmail("test@example.com");
36        verify(usuarioRepository).save(usuario);
37    }
38
39    @ParameterizedTest
40    @ValueSource(strings = {"", " ", "a", "ab"})
41    @DisplayName("Deber a fallar con nombres inv lidos")
42    void validarNombre_NombresInvalidos_DeberiaFallar(String nombre) {
43        Usuario usuario = new Usuario(nombre, "test@example.com", "123")
44    ;
45
46        assertThrows(ValidationException.class,
47            () -> usuarioService.crearUsuario(usuario));
48    }
49 }

```

Listing 5.7: Ejemplo de Test con JUnit 5 y Mockito

5.3.2. Tests de Integración con Spring Boot

```
1 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.  
    RANDOM_PORT)  
2 @TestPropertySource(locations = "classpath:application-test.properties")  
3 @Transactional  
4 class UsuarioIntegrationTest {  
5  
6     @Autowired  
7     private TestRestTemplate restTemplate;  
8  
9     @Autowired  
10    private UsuarioRepository usuarioRepository;  
11  
12    @Test  
13    void flujoCompletoUsuario_DeberiaFuncionar() {  
14        // Crear usuario  
15        Usuario nuevoUsuario = new Usuario("Juan", "juan@test.com", "123  
16    ");  
17        ResponseEntity<Usuario> createResponse = restTemplate.  
18    postForEntity(  
19        "/api/usuarios", nuevoUsuario, Usuario.class);  
20  
21        assertThat(createResponse.getStatusCode()).isEqualTo(HttpStatus.  
22    CREATED);  
23        assertThat(createResponse.getBody()).isNotNull();  
24  
25        Long usuarioId = createResponse.getBody().getId();  
26  
27        // Obtener usuario  
28        ResponseEntity<Usuario> getResponse = restTemplate.getForEntity(  
29        "/api/usuarios/" + usuarioId, Usuario.class);  
30  
31        assertThat(getResponse.getStatusCode()).isEqualTo(HttpStatus.OK)  
32    ;  
33        assertThat(getResponse.getBody().getNombre()).isEqualTo("Juan");  
34  
35        // Verificar en base de datos  
36        Optional<Usuario> usuarioDb = usuarioRepository.findById(  
37    usuarioId);  
38        assertThat(usuarioDb).isPresent();  
39        assertThat(usuarioDb.get().getEmail()).isEqualTo("juan@test.com")  
40    );  
41    }
```

Listing 5.8: Test de Integración Completo

5.4. Pipeline Java Optimizado

Optimizaciones para Pipelines Java

- Usar cache de dependencias Maven
- Paralelizar análisis de código
- Ejecutar tests en paralelo por módulos
- Usar perfiles Maven según ambiente
- Configurar timeouts apropiados

```

1 pipeline {
2   agent any
3
4   tools {
5     maven 'Maven-3.8.6'
6     jdk 'JDK-11'
7   }
8
9   stages {
10    stage('Build with Cache') {
11      steps {
12        // Cache de repositorio Maven
13        cache(maxCacheSize: 250, caches: [
14          arbitraryFileCache(
15            path: '.m2/repository',
16            includes: '**/*',
17            fingerprinting: true
18          )
19        ]) {
20          sh '''
21            mvn clean compile \
22              -Dmaven.repo.local=.m2/repository \
23              -DskipTests=true
24          '''
25        }
26      }
27    }
28
29    stage('Parallel Quality Checks') {
30      parallel {
31        stage('Unit Tests') {
32          steps {
33            sh 'mvn test -Dmaven.repo.local=.m2/repository'
34          }
35        }
36        stage('Checkstyle') {
37          steps {
38            sh 'mvn checkstyle:check -Dmaven.repo.local=.m2/
39            repository'
40          }
41        }
42        stage('SpotBugs') {
43          steps {

```



```
43 repository' sh 'mvn spotbugs:check -Dmaven.repo.local=.m2/
44 }
45 }
46 }
47 }
48 }
49 }
```

Listing 5.9: Pipeline con Cache de Maven

Capítulo 6

Plugins Esenciales

6.1. Plugins Específicos para Java

6.1.1. Maven Integration Plugin

- **Funcionalidad:** Integración nativa con Maven
- **Características:** Jobs Maven, dependencias automáticas
- **Configuración:** Automática con Maven tools

6.1.2. JaCoCo Plugin

```
1 post {  
2     always {  
3         publishCoverage adapters: [  
4             jacocoAdapter('target/site/jacoco/jacoco.xml')  
5         ], sourceFileResolver: sourceFiles('STORE_LAST_BUILD')  
6     }  
7 }
```

Listing 6.1: Publicar Cobertura JaCoCo

6.1.3. Warnings Next Generation Plugin

```
1 post {  
2     always {  
3         recordIssues enabledForFailure: true, tools: [  
4             checkStyle(),  
5             spotBugs(),  
6             pmd(),  
7             mavenConsole()  
8         ]  
9     }  
10 }
```

Listing 6.2: Análisis de Warnings

6.2. Plugins de Gestión de Código

6.2.1. Git Plugin

- **Funcionalidad:** Integración con repositorios Git
- **Instalación:** Incluido por defecto
- **Uso:** Checkout automático, webhooks, multi-branch

6.2.2. GitHub Integration Plugin

```
1 pipeline {
2   agent any
3
4   triggers {
5     githubPush()
6   }
7
8   stages {
9     stage('Build on Push') {
10      steps {
11        echo 'Triggered by GitHub push event'
12      }
13    }
14  }
15 }
```

Listing 6.3: Configuración GitHub Webhook

6.3. Plugins de Testing

6.3.1. JUnit Plugin

```
1 post {
2   always {
3     junit testResults: '**/target/surefire-reports/*.xml',
4           allowEmptyResults: true
5   }
6 }
```

Listing 6.4: Publicar Resultados JUnit

6.3.2. Code Coverage Plugin

```
1 steps {
2   sh 'npm run test:coverage'
3   publishHTML([
4     allowMissing: false,
5     alwaysLinkToLastBuild: true,
6     keepAll: true,
7     reportDir: 'coverage',
8     reportFiles: 'index.html',
```

```
9     reportName: 'Coverage Report'
10   })
11 }
```

Listing 6.5: Cobertura de Código

6.4. Plugins de Despliegue

6.4.1. Docker Pipeline Plugin

```
1 pipeline {
2   agent any
3
4   environment {
5     DOCKER_REGISTRY = 'your-registry.com'
6     IMAGE_NAME = 'mi-app'
7   }
8
9   stages {
10    stage('Build Docker Image') {
11      steps {
12        script {
13          def image = docker.build("${DOCKER_REGISTRY}/${
14            IMAGE_NAME}:${BUILD_NUMBER}")
15        }
16      }
17
18      stage('Push to Registry') {
19        steps {
20          script {
21            docker.withRegistry("https://${DOCKER_REGISTRY}", '
22            docker-registry-credentials') {
23              def image = docker.image("${DOCKER_REGISTRY}/${
24                IMAGE_NAME}:${BUILD_NUMBER}")
25              image.push()
26              image.push("latest")
27            }
28          }
29        }
30      }
31    }
32  }
33 }
```

Listing 6.6: Build y Push Docker Image

6.4.2. Kubernetes Plugin

```
1 stage('Deploy to Kubernetes') {
2   steps {
3     kubernetesDeploy(
4       configs: 'k8s-deployment.yaml',
5       kubeconfigId: 'k8s-config'
6     )
7   }
8 }
```

8 }

Listing 6.7: Despliegue en Kubernetes

Capítulo 7

Proyecto Práctico: Aplicación Java Spring Boot

7.1. Estructura del Proyecto

Vamos a crear un proyecto completo de ejemplo que incluya:

- Aplicación Java Spring Boot con REST API
- Tests unitarios e integración con JUnit 5 y Mockito
- Análisis de código con Checkstyle, SpotBugs y SonarQube
- Pipeline completo de CI/CD con Maven
- Despliegue con Docker y Kubernetes
- Cobertura de código con JaCoCo

7.2. Configuración del Proyecto Maven

El proyecto utiliza Maven como herramienta de build management. El `pom.xml` incluye:

```
1 <project>
2   <groupId>com.ejemplo.jenkins</groupId>
3   <artifactId>demo-java-app</artifactId>
4   <version>1.0.0-SNAPSHOT</version>
5   <packaging>jar</packaging>
6
7   <parent>
8     <groupId>org.springframework.boot</groupId>
9     <artifactId>spring-boot-starter-parent</artifactId>
10    <version>2.7.2</version>
11  </parent>
12
13  <dependencies>
14    <dependency>
15      <groupId>org.springframework.boot</groupId>
16      <artifactId>spring-boot-starter-web</artifactId>
17    </dependency>
18    <dependency>
```

```

19         <groupId>org.springframework.boot</groupId>
20         <artifactId>spring-boot-starter-data-jpa</artifactId>
21     </dependency>
22     <!-- Tests -->
23     <dependency>
24         <groupId>org.springframework.boot</groupId>
25         <artifactId>spring-boot-starter-test</artifactId>
26         <scope>test</scope>
27     </dependency>
28 </dependencies>
29 </project>

```

Listing 7.1: Estructura del pom.xml

7.3. Aplicación de Ejemplo

La aplicación de demostración es una REST API para gestión de usuarios:

```

1 @Entity
2 @Table(name = "usuarios")
3 public class Usuario {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @NotBlank(message = "El nombre es obligatorio")
9     @Size(min = 2, max = 50)
10    private String nombre;
11
12    @NotBlank(message = "El email es obligatorio")
13    @Email(message = "El formato del email no es v lido")
14    private String email;
15
16    @NotBlank(message = "El tel fono es obligatorio")
17    private String telefono;
18
19    // Constructor, getters y setters
20 }

```

Listing 7.2: Entidad Usuario

```

1 @RestController
2 @RequestMapping("/api/usuarios")
3 public class UsuarioController {
4
5     private final UsuarioService usuarioService;
6
7     @GetMapping
8     public ResponseEntity<List<Usuario>> obtenerTodosLosUsuarios() {
9         List<Usuario> usuarios = usuarioService.obtenerTodosLosUsuarios
10        ();
11         return ResponseEntity.ok(usuarios);
12     }
13
14     @PostMapping
15     public ResponseEntity<Usuario> crearUsuario(@Valid @RequestBody
16        Usuario usuario) {
17         try {

```



```

16         Usuario nuevoUsuario = usuarioService.crearUsuario(usuario);
17         return ResponseEntity.status(HttpStatus.CREATED).body(
nuevoUsuario);
18     } catch (IllegalArgumentException e) {
19         return ResponseEntity.status(HttpStatus.CONFLICT).build();
20     }
21 }
22
23 @GetMapping("/health")
24 public ResponseEntity<Object> healthCheck() {
25     return ResponseEntity.ok(Map.of(
26         "status", "OK",
27         "timestamp", Instant.now().toString(),
28         "service", "usuario-service"
29     ));
30 }
31 }

```

Listing 7.3: Controlador REST

7.4. Tests con JUnit 5

7.4.1. Tests Unitarios

```

1 @ExtendWith(MockitoExtension.class)
2 class UsuarioServiceTest {
3
4     @Mock
5     private UsuarioRepository usuarioRepository;
6
7     @InjectMocks
8     private UsuarioService usuarioService;
9
10    @Test
11    void crearUsuario_DeberiaCrearUsuarioExitosamente() {
12        // Given
13        Usuario nuevoUsuario = new Usuario("Mar a", "maria@example.com",
14        "123");
15        when(usuarioRepository.existsByEmail("maria@example.com"))
16        .thenReturn(false);
17        when(usuarioRepository.save(nuevoUsuario)).thenReturn(
18        nuevoUsuario);
19
20        // When
21        Usuario resultado = usuarioService.crearUsuario(nuevoUsuario);
22
23        // Then
24        assertNotNull(resultado);
25        assertEquals("Mar a", resultado.getNombre());
26        verify(usuarioRepository).existsByEmail("maria@example.com");
27        verify(usuarioRepository).save(nuevoUsuario);
28    }
29
30    @Test
31    void crearUsuario_DeberiaLanzarExcepcionSiEmailExiste() {
32        // Given

```

```

31     Usuario usuario = new Usuario("Juan", "juan@example.com", "123")
32     ;
33     when(usuarioRepository.existsByEmail("juan@example.com"))
34         .thenReturn(true);
35
36     // When & Then
37     IllegalArgumentException exception = assertThrows(
38         IllegalArgumentException.class,
39         () -> usuarioService.crearUsuario(usuario)
40     );
41     assertEquals("Ya existe un usuario con este email", exception.
42         getMessage());
43 }

```

Listing 7.4: Test Unitario del Service

7.4.2. Tests de Integración

```

1 @WebMvcTest(UsuarioController.class)
2 class UsuarioControllerTest {
3
4     @Autowired
5     private MockMvc mockMvc;
6
7     @MockBean
8     private UsuarioService usuarioService;
9
10    @Test
11    void crearUsuario_DeberiaCrearUsuarioExitosamente() throws Exception
12    {
13        // Given
14        Usuario usuario = new Usuario("Pedro", "pedro@example.com", "555
15        ");
16        usuario.setId(1L);
17        when(usuarioService.crearUsuario(any())).thenReturn(usuario);
18
19        // When & Then
20        mockMvc.perform(post("/api/usuarios")
21            .contentType(MediaType.APPLICATION_JSON)
22            .content(objectMapper.writeValueAsString(usuario))
23            .andExpect(status().isCreated())
24            .andExpect(jsonPath("$.nombre").value("Pedro"))
25            .andExpect(jsonPath("$.email").value("pedro@example.com"
26        ));
27    }
28 }

```

Listing 7.5: Test de Integración con MockMvc

7.5. Pipeline de Jenkins para Java

El pipeline completo para la aplicación Java incluye múltiples etapas optimizadas:

```

1 pipeline {
2     agent any

```

```

3
4  tools {
5      maven 'Maven-3.8.6'
6      jdk 'JDK-11'
7  }
8
9  environment {
10     MAVEN_OPTS = '-Xmx1024m'
11     DOCKER_REGISTRY = 'localhost:5000'
12     APP_NAME = 'demo-java-app'
13     SONAR_PROJECT_KEY = 'demo-java-app'
14     SPRING_PROFILES_ACTIVE = "${env.BRANCH_NAME == 'main' ? 'prod' :
'dev'}"
15 }
16
17 stages {
18     stage('Checkout') {
19         steps {
20             checkout scm
21             script {
22                 env.GIT_COMMIT_SHORT = sh(
23                     script: 'git rev-parse --short HEAD',
24                     returnStdout: true
25                 ).trim()
26             }
27         }
28     }
29
30     stage('Install Dependencies') {
31         steps {
32             sh 'npm ci'
33         }
34     }
35
36     stage('Code Quality') {
37         parallel {
38             stage('Lint') {
39                 steps {
40                     sh 'npm run lint'
41                     recordIssues enabledForFailure: true, tools: [
42                         eslint()
43                     ]
44                 }
45             }
46             stage('Security Audit') {
47                 steps {
48                     sh 'npm audit --audit-level high'
49                 }
50             }
51         }
52
53     stage('Test') {
54         steps {
55             sh 'npm run test:coverage'
56         }
57         post {
58             always {

```

```

59         publishTestResults testResultsPattern: 'test-results
.xml'
60         publishCoverage adapters: [
61             istanbulCoberturaAdapter('coverage/cobertura-
coverage.xml')
62         ], sourceFileResolver: sourceFiles('STORE_LAST_BUILD
')
63     }
64 }
65 }
66
67 stage('SonarQube Analysis') {
68     when {
69         anyOf {
70             branch 'main'
71             branch 'develop'
72         }
73     }
74     steps {
75         withSonarQubeEnv('SonarQube') {
76             sh '''
77                 sonar-scanner \
78                 -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
79                 -Dsonar.sources=src \
80                 -Dsonar.tests=tests \
81                 -Dsonar.javascript.lcov.reportPaths=coverage/
lcov.info \
82                 -Dsonar.testExecutionReportPaths=test-results.
xml
83             '''
84         }
85     }
86 }
87
88 stage('Quality Gate') {
89     when {
90         anyOf {
91             branch 'main'
92             branch 'develop'
93         }
94     }
95     steps {
96         timeout(time: 1, unit: 'HOURS') {
97             waitForQualityGate abortPipeline: true
98         }
99     }
100 }
101
102 stage('Build') {
103     steps {
104         sh 'npm run build'
105         archiveArtifacts artifacts: 'dist/**/*',
allowEmptyArchive: false
106     }
107 }
108
109 stage('Build Docker Image') {
110     steps {

```

```

111         script {
112             env.IMAGE_TAG = "${BUILD_NUMBER}-${env.
GIT_COMMIT_SHORT}"
113             def image = docker.build("${DOCKER_REGISTRY}/${
APP_NAME}:${env.IMAGE_TAG}")
114             env.DOCKER_IMAGE = image.id
115         }
116     }
117 }
118
119 stage('Security Scan Docker Image') {
120     steps {
121         sh "docker run --rm -v /var/run/docker.sock:/var/run/
docker.sock aquasec/trivy image ${DOCKER_REGISTRY}/${APP_NAME}:${env.
IMAGE_TAG}"
122     }
123 }
124
125 stage('Push Docker Image') {
126     when {
127         anyOf {
128             branch 'main'
129             branch 'develop'
130         }
131     }
132     steps {
133         script {
134             docker.withRegistry("http://${DOCKER_REGISTRY}") {
135                 def image = docker.image("${DOCKER_REGISTRY}/${
APP_NAME}:${env.IMAGE_TAG}")
136                 image.push()
137                 if (env.BRANCH_NAME == 'main') {
138                     image.push('latest')
139                 }
140             }
141         }
142     }
143 }
144
145 stage('Deploy to Staging') {
146     when {
147         branch 'develop'
148     }
149     steps {
150         script {
151             sh """
152                 docker-compose -f docker-compose.staging.yml
down
153                 export IMAGE_TAG=${env.IMAGE_TAG}
154                 docker-compose -f docker-compose.staging.yml up
-d
155                 """
156         }
157
158         // Health check
159         sh '''
160             for i in {1..30}; do

```

```

161         if curl -f http://staging.localhost:3000/health;
162     then
163         echo "Staging deployment successful!"
164         break
165     fi
166     echo "Waiting for staging deployment... ($i/30)"
167     sleep 10
168 done
169 '''
170 }
171
172 stage('Integration Tests') {
173     when {
174         branch 'develop'
175     }
176     steps {
177         sh 'npm run test:integration -- --baseUrl=http://staging
178 .localhost:3000'
179     }
180
181     stage('Deploy to Production') {
182         when {
183             branch 'main'
184         }
185         steps {
186             script {
187                 // Manual approval for production
188                 def deploy = input(
189                     message: 'Deploy to Production?',
190                     parameters: [
191                         choice(
192                             name: 'DEPLOY_ENVIRONMENT',
193                             choices: ['production', 'canary'],
194                             description: 'Select deployment strategy
195 ,
196
197                     )
198                 ]
199             )
200
201             if (deploy == 'canary') {
202                 sh """
203                 kubectl set image deployment/nodejs-demo-app
204 -canary \
205                 app=${DOCKER_REGISTRY}/${APP_NAME}:${env.
206 IMAGE_TAG} \
207                 -n production
208                 kubectl rollout status deployment/nodejs-
209 demo-app-canary -n production
210                 """
211
212                 // Wait for canary analysis
213                 sleep(time: 300, unit: 'SECONDS')
214
215                 // Promote to full production
216                 sh """

```

```

212         kubect1 set image deployment/nodejs-demo-app
213         \
214         app=${DOCKER_REGISTRY}/${APP_NAME}:${env.
215         IMAGE_TAG} \
216         -n production
217         kubect1 rollout status deployment/nodejs-
218         demo-app -n production
219         """
220         } else {
221         sh """
222         kubect1 set image deployment/nodejs-demo-app
223         \
224         app=${DOCKER_REGISTRY}/${APP_NAME}:${env.
225         IMAGE_TAG} \
226         -n production
227         kubect1 rollout status deployment/nodejs-
228         demo-app -n production
229         """
230         }
231     }
232 }
233
234 post {
235     always {
236         // Cleanup
237         sh 'docker system prune -f'
238         cleanWs()
239     }
240
241     success {
242         script {
243             if (env.BRANCH_NAME == 'main') {
244                 slackSend(
245                     channel: '#deployments',
246                     color: 'good',
247                     message: """
248                     *Production Deployment Successful*
249                     Project: ${APP_NAME}
250                     Version: ${env.IMAGE_TAG}
251                     Duration: ${currentBuild.durationString}
252                     URL: https://production.yourapp.com
253                     """
254                 )
255             }
256         }
257     }
258
259     failure {
260         slackSend(
261             channel: '#ci-cd-alerts',
262             color: 'danger',
263             message: """
264             *Build Failed*
265             Project: ${APP_NAME}
266             Branch: ${env.BRANCH_NAME}
267             Build: ${BUILD_NUMBER}

```

```

264     Duration: ${currentBuild.durationString}
265     Check: ${BUILD_URL}
266         ""
267     )
268 }
269
270     unstable {
271         slackSend(
272             channel: '#ci-cd-alerts',
273             color: 'warning',
274             message: ""
275         *Build Unstable*
276         Project: ${APP_NAME}
277         Branch: ${env.BRANCH_NAME}
278         Build: ${BUILD_NUMBER}
279         Check: ${BUILD_URL}
280             ""
281         )
282     }
283 }
284 }

```

Listing 7.6: Jenkinsfile Completo para Java

Capítulo 8

Configuración Avanzada

8.1. Jenkins como Código (JCasC)

Jenkins Configuration as Code permite gestionar la configuración de Jenkins mediante archivos YAML:

```
1 jenkins:
2   systemMessage: "Jenkins configurado mediante JCasC"
3   numExecutors: 2
4   scmCheckoutRetryCount: 3
5   mode: NORMAL
6
7   securityRealm:
8     local:
9       allowsSignup: false
10      users:
11        - id: "admin"
12          password: "${JENKINS_ADMIN_PASSWORD}"
13        - id: "developer"
14          password: "${JENKINS_DEV_PASSWORD}"
15
16   authorizationStrategy:
17     globalMatrix:
18       permissions:
19         - "Overall/Administer:admin"
20         - "Overall/Read:authenticated"
21         - "Job/Build:developer"
22         - "Job/Cancel:developer"
23         - "Job/Read:developer"
24
25   clouds:
26     - docker:
27       name: "docker"
28       dockerApi:
29         dockerHost:
30           uri: "unix:///var/run/docker.sock"
31       templates:
32         - labelString: "docker-agent"
33           dockerTemplateBase:
34             image: "jenkins/inbound-agent"
35             remoteFs: "/home/jenkins/agent"
36
37 security:
```

```

38 globalJobDslSecurityConfiguration:
39   useScriptSecurity: false
40
41 credentials:
42   system:
43     domainCredentials:
44       - credentials:
45         - usernamePassword:
46           scope: GLOBAL
47           id: "github-credentials"
48           username: "${GITHUB_USERNAME}"
49           password: "${GITHUB_TOKEN}"
50         - basicSSHUserPrivateKey:
51           scope: GLOBAL
52           id: "ssh-key"
53           username: "jenkins"
54           privateKeySource:
55             directEntry:
56               privateKey: "${SSH_PRIVATE_KEY}"
57
58 jobs:
59   - script: |
60     multibranchPipelineJob('nodejs-demo-app') {
61       branchSources {
62         git {
63           id('nodejs-demo-app')
64           remote('https://github.com/usuario/nodejs-demo-app.git
65         ')
66           credentialsId('github-credentials')
67         }
68       }
69       orphanedItemStrategy {
70         discardOldItems {
71           numToKeep(10)
72         }
73       }
74       triggers {
75         periodic(5)
76       }

```

Listing 8.1: jenkins.yaml - Configuración como Código

8.2. Shared Libraries

Las librerías compartidas permiten reutilizar código entre pipelines:

```

1 def call(Map config) {
2   script {
3     // Validar par metros requeridos
4     if (!config.namespace) {
5       error("namespace is required")
6     }
7     if (!config.deployment) {
8       error("deployment name is required")
9     }
10    if (!config.image) {

```

```

11         error("image is required")
12     }
13
14     // Desplegar a Kubernetes
15     withKubeConfig([credentialsId: config.kubeconfig ?: 'k8s-config
16     ']) {
17         sh """
18             kubectl set image deployment/${config.deployment} \
19             ${config.container ?: 'app'}=${config.image} \
20             -n ${config.namespace}
21
22             kubectl rollout status deployment/${config.deployment} \
23             -n ${config.namespace} \
24             --timeout=${config.timeout ?: '300s'}
25         """
26
27         if (config.healthCheck) {
28             // Verificar health check
29             sh """
30                 for i in {1..30}; do
31                     if kubectl get pods -n ${config.namespace} \
32                     -l app=${config.deployment} \
33                     -o jsonpath='{.items[0].status.phase}' | grep
Running; then
34                         echo "Deployment successful!"
35                         break
36                     fi
37                     echo "Waiting for pods to be ready... (\$i/30)"
38                     sleep 10
39                 done
40             """
41         }
42     }
43 }

```

Listing 8.2: vars/deployToKubernetes.groovy

```

1 @Library('jenkins-shared-library') _
2
3 pipeline {
4     agent any
5
6     stages {
7         stage('Deploy') {
8             steps {
9                 deployToKubernetes([
10                     namespace: 'production',
11                     deployment: 'nodejs-demo-app',
12                     image: "${DOCKER_REGISTRY}/${APP_NAME}:${BUILD_NUMBER}",
13                     container: 'app',
14                     timeout: '600s',
15                     healthCheck: true,
16                     kubeconfig: 'k8s-production-config'
17                 ])
18             }
19         }
20     }
21 }

```

21 }

Listing 8.3: Uso de Shared Library en Pipeline

Capítulo 9

Monitoreo y Observabilidad

9.1. Métricas de Jenkins

9.1.1. Plugin de Métricas

```
1 // En el pipeline
2 pipeline {
3     agent any
4
5     stages {
6         stage('Collect Metrics') {
7             steps {
8                 script {
9                     // M tricas personalizadas
10                    def buildDuration = currentBuild.duration
11                    def testResults = junit testResultsPattern: 'test-
12                    results.xml'
13
14                    // Enviar a sistema de m tricas (InfluxDB,
15                    Prometheus)
16                    sh """
17                        curl -i -XPOST 'http://influxdb:8086/write?db=
18                        jenkins' \
19                        --data-binary '
20                        build_metrics ,job=${JOB_NAME},build=${
21                        BUILD_NUMBER} \
22                        duration=${buildDuration},
23                        tests_total=${testResults.totalCount},
24                        tests_failed=${testResults.failCount},
25                        tests_passed=${testResults.passCount}
26                        ,
27                        """
28                }
            }
        }
    }
}
```

Listing 9.1: Configuración de Métricas

9.2. Alertas y Notificaciones

9.2.1. Integración con Slack

```

1 def sendSlackNotification(String status, String color) {
2     def message = [
3         channel: '#ci-cd',
4         color: color,
5         message: ""
6     ]
7     *${status.toUpperCase()}*: ${env.JOB_NAME} - ${env.BUILD_NUMBER}
8     *Branch*: ${env.BRANCH_NAME}
9     *Commit*: ${env.GIT_COMMIT?.take(7)}
10    *Author*: ${env.CHANGE_AUTHOR ?: 'Unknown'}
11    *Duration*: ${currentBuild.durationString}
12
13    *Changes*:
14    ${getChangeString()}
15
16    *Console Output*: ${BUILD_URL}console
17    "",
18    iconEmoji: status == 'SUCCESS' ? ':white_check_mark:' : ':x:'
19    ]
20
21    slackSend(message)
22 }
23
24 def getChangeString() {
25     def changeLogSets = currentBuild.changeSets
26     def changes = []
27
28     for (int i = 0; i < changeLogSets.size(); i++) {
29         def entries = changeLogSets[i].items
30         for (int j = 0; j < entries.length; j++) {
31             def entry = entries[j]
32             changes.add("    ${entry.msg} - ${entry.author}")
33         }
34     }
35
36     return changes.size() > 0 ? changes.join('\n') : 'No changes'
37 }
38
39 // Uso en post actions
40 post {
41     success {
42         sendSlackNotification('SUCCESS', 'good')
43     }
44     failure {
45         sendSlackNotification('FAILURE', 'danger')
46     }
47 }

```

Listing 9.2: Notificaciones Avanzadas con Slack

Capítulo 10

Mejores Prácticas

10.1. Organización de Pipelines

Estructura Recomendada

- **Multibranch Pipelines:** Para proyectos con múltiples ramas
- **Organization Folders:** Para múltiples repositorios
- **Shared Libraries:** Para código reutilizable
- **Pipeline Templates:** Para estandarizar procesos

10.1.1. Convenciones de Nomenclatura

```
1 Organization/  
2     Frontend/  
3         web-app-main  
4         web-app-develop  
5         mobile-app  
6     Backend/  
7         api-service-main  
8         api-service-develop  
9         auth-service  
10    Infrastructure/  
11        terraform-deployments  
12        ansible-configurations
```

Listing 10.1: Estructura de Jobs Recomendada

10.2. Seguridad

10.2.1. Gestión de Secretos

```
1 pipeline {  
2     agent any  
3  
4     environment {  
5         // Variables p blicas
```

```

6     APP_NAME = 'mi-aplicacion'
7     ENVIRONMENT = 'production'
8 }
9
10 stages {
11     stage('Deploy') {
12         steps {
13             // Credenciales seguras
14             withCredentials([
15                 string(credentialsId: 'api-key', variable: 'API_KEY
16             ),
17                 usernamePassword(
18                     credentialsId: 'db-credentials',
19                     usernameVariable: 'DB_USER',
20                     passwordVariable: 'DB_PASS'
21                 ),
22                 file(credentialsId: 'kubeconfig', variable: '
KUBECONFIG_FILE')
23             ]) {
24                 sh '''
25                     # Las variables est n disponibles solo en este
26                     bloque
27                     echo "Deploying with API key: ${API_KEY
28                     :0:4}****"
29                     kubectl --kubeconfig="$KUBECONFIG_FILE" apply -f
30                     deployment.yaml
31                 '''
32             }
33         }
34     }
35 }

```

Listing 10.2: Uso Seguro de Credenciales

10.2.2. Script Approval

Seguridad de Scripts

- Activar Script Security para pipelines
- Revisar y aprobar scripts manualmente
- Usar Shared Libraries para código confiable
- Limitar permisos de usuarios

10.3. Performance y Optimización

10.3.1. Paralelización

```

1 stage('Quality Checks') {
2     parallel {
3         stage('Unit Tests') {

```



```
4         steps {
5             sh 'npm run test:unit'
6         }
7     }
8     stage('Integration Tests') {
9         steps {
10             sh 'npm run test:integration'
11         }
12     }
13     stage('Lint') {
14         steps {
15             sh 'npm run lint'
16         }
17     }
18     stage('Security Scan') {
19         steps {
20             sh 'npm audit'
21         }
22     }
23 }
24 }
```

Listing 10.3: Optimización con Paralelización

10.3.2. Cacheo y Optimización

```
1 pipeline {
2     agent any
3
4     stages {
5         stage('Install Dependencies') {
6             steps {
7                 // Cache de node_modules
8                 cache(maxCacheSize: 250, caches: [
9                     arbitraryFileCache(
10                        path: 'node_modules',
11                        includes: '**/*',
12                        fingerprinting: true
13                    )
14                ]) {
15                    sh '''
16                        if [ ! -d "node_modules" ] || [ package-lock.
17                        json -nt node_modules ]; then
18                            npm ci
19                        else
20                            echo "Using cached node_modules"
21                        fi
22                    '''
23                }
24            }
25        }
26    }
```

Listing 10.4: Uso de Cache

Capítulo 11

Troubleshooting y Depuración

11.1. Problemas Comunes

11.1.1. Problemas de Conexión

Diagnóstico de Conexiones

- Verificar conectividad de red
- Revisar configuración de proxy
- Validar certificados SSL
- Comprobar firewall y puertos

```
1 #!/bin/bash
2 # Diagn stico de conectividad
3
4 echo "=== Diagn stico de Jenkins ==="
5 echo "Fecha: $(date)"
6 echo "Usuario: $(whoami)"
7 echo "Java Version: $(java -version 2>&1 | head -n 1)"
8
9 # Test de conectividad
10 echo -e "\n=== Test de Conectividad ==="
11 ping -c 3 github.com
12 curl -I https://github.com
13
14 # Verificar puertos
15 echo -e "\n=== Puertos Jenkins ==="
16 netstat -tlnp | grep :8080
17 netstat -tlnp | grep :50000
18
19 # Espacio en disco
20 echo -e "\n=== Espacio en Disco ==="
21 df -h
22
23 # Memoria
24 echo -e "\n=== Memoria ==="
25 free -h
26
27 # Logs de Jenkins
```

```
28 echo -e "\n=== últimos logs de Jenkins ==="
29 tail -n 20 /var/log/jenkins/jenkins.log
```

Listing 11.1: Scripts de Diagnóstico

11.1.2. Depuración de Pipelines

```
1 pipeline {
2   agent any
3
4   options {
5     // Mantener logs por m s tiempo
6     buildDiscarder(logRotator(numToKeepStr: '50'))
7     // Timeout global
8     timeout(time: 1, unit: 'HOURS')
9     // Timestamps en logs
10    timestamps()
11  }
12
13  stages {
14    stage('Debug Info') {
15      steps {
16        script {
17          echo "=== DEBUG INFORMATION ==="
18          echo "Job Name: ${env.JOB_NAME}"
19          echo "Build Number: ${env.BUILD_NUMBER}"
20          echo "Branch: ${env.BRANCH_NAME}"
21          echo "Workspace: ${env.WORKSPACE}"
22          echo "Node: ${env.NODE_NAME}"
23
24          // Variables de entorno
25          echo "=== ENVIRONMENT VARIABLES ==="
26          sh 'env | sort'
27
28          // Información del sistema
29          echo "=== SYSTEM INFO ==="
30          sh 'uname -a'
31          sh 'pwd'
32          sh 'ls -la'
33        }
34      }
35    }
36
37    stage('Build with Error Handling') {
38      steps {
39        script {
40          try {
41            sh 'npm run build'
42          } catch (Exception e) {
43            echo "Build failed: ${e.getMessage()}"
44            // Collect debug info
45            sh 'npm --version'
46            sh 'node --version'
47            sh 'cat package.json'
48            throw e
49          }
50        }
51      }
52    }
53  }
54}
```

```

51     }
52   }
53 }
54
55 post {
56   failure {
57     script {
58       // Collect debug artifacts on failure
59       archiveArtifacts artifacts: 'npm-debug.log*',
allowEmptyArchive: true
60       archiveArtifacts artifacts: '.npm/_logs/*',
allowEmptyArchive: true
61
62       // Send detailed error info
63       def errorLog = sh(
64         script: 'tail -n 100 ${BUILD_URL}consoleText',
65         returnStdout: true
66       )
67
68       slackSend(
69         channel: '#debug',
70         color: 'danger',
71         message: ""
72 Pipeline Failed: ${env.JOB_NAME} - ${env.BUILD_NUMBER}
73 Error Log:
74 '''
75 ${errorLog}
76 '''
77
78         """
79       )
80     }
81   }
82 }

```

Listing 11.2: Pipeline con Depuración

11.2. Logs y Monitoreo

11.2.1. Configuración de Logs

```

1 <configuration>
2   <appender name="FILE" class="ch.qos.logback.core.rolling.
RollingFileAppender">
3     <file>/var/log/jenkins/jenkins.log</file>
4     <rollingPolicy class="ch.qos.logback.core.rolling.
TimeBasedRollingPolicy">
5       <fileNamePattern>/var/log/jenkins/jenkins.%d{yyyy-MM-dd}.log
</fileNamePattern>
6       <maxHistory>30</maxHistory>
7     </rollingPolicy>
8     <encoder>
9       <pattern>%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger
{36} - %msg%n</pattern>
10     </encoder>
11   </appender>

```

```

12
13     <logger name="hudson.model.Run" level="DEBUG"/>
14     <logger name="jenkins.scm" level="DEBUG"/>
15     <logger name="org.jenkinsci.plugins.workflow" level="DEBUG"/>
16
17     <root level="INFO">
18         <appender-ref ref="FILE"/>
19     </root>
20 </configuration>

```

Listing 11.3: logback.xml - Configuración de Logging

Capítulo 12

Casos de Uso Java Específicos

12.1. Gestión de Artefactos Maven

12.1.1. Nexus Repository Manager

Nexus es esencial para gestionar artefactos Maven en entornos empresariales:

```
1 <distributionManagement>
2   <repository>
3     <id>nexus-releases</id>
4     <name>Nexus Release Repository</name>
5     <url>http://nexus.company.com:8081/repository/maven-releases/</
6   url>
7   </repository>
8   <snapshotRepository>
9     <id>nexus-snapshots</id>
10    <name>Nexus Snapshot Repository</name>
11    <url>http://nexus.company.com:8081/repository/maven-snapshots/</
12  url>
13 </snapshotRepository>
14 </distributionManagement>
```

Listing 12.1: Configuración de Nexus en Maven

```
1 stage('Deploy to Nexus') {
2   when {
3     branch 'main'
4   }
5   steps {
6     configFileProvider([
7       configFile(fileId: 'maven-settings', variable: '
8     MAVEN_SETTINGS')
9     ]) {
10      sh """
11        mvn deploy \
12        -s $MAVEN_SETTINGS \
13        -DskipTests=true \
14        -Dspring.profiles.active=prod
15      """
16    }
17  }
```

Listing 12.2: Pipeline con Deploy a Nexus

12.2. Análisis de Cobertura Avanzado

12.2.1. Múltiples Tipos de Tests

```

1 <plugin>
2   <groupId>org.jacoco</groupId>
3   <artifactId>jacoco-maven-plugin</artifactId>
4   <version>0.8.7</version>
5   <executions>
6     <!-- Unit Test Coverage -->
7     <execution>
8       <id>prepare-agent</id>
9       <goals>
10        <goal>prepare-agent</goal>
11      </goals>
12    </execution>
13
14    <!-- Integration Test Coverage -->
15    <execution>
16      <id>prepare-agent-integration</id>
17      <goals>
18        <goal>prepare-agent-integration</goal>
19      </goals>
20    </execution>
21
22    <!-- Merge reports -->
23    <execution>
24      <id>merge-reports</id>
25      <phase>verify</phase>
26      <goals>
27        <goal>merge</goal>
28      </goals>
29      <configuration>
30        <destFile>${project.build.directory}/jacoco-merged.exec<
31      /destFile>
32        <fileSets>
33          <fileSet>
34            <directory>${project.build.directory}</directory>
35          </fileSet>
36          <includes>
37            <include>jacoco.exec</include>
38            <include>jacoco-it.exec</include>
39          </includes>
40        </fileSet>
41      </fileSets>
42    </configuration>
43  </execution>
44
45  <!-- Generate merged report -->
46  <execution>
47    <id>report-merged</id>
48    <phase>verify</phase>
49    <goals>
50      <goal>report</goal>
51    </goals>
52    <configuration>
53      <dataFile>${project.build.directory}/jacoco-merged.exec<
54    /dataFile>

```



```

52         <outputDirectory>${project.reporting.outputDirectory}/
jacoco-merged</outputDirectory>
53     </configuration>
54 </execution>
55
56 <!-- Quality gate -->
57 <execution>
58     <id>check-coverage</id>
59     <goals>
60         <goal>check</goal>
61     </goals>
62     <configuration>
63         <rules>
64             <rule>
65                 <element>BUNDLE</element>
66                 <limits>
67                     <limit>
68                         <counter>INSTRUCTION</counter>
69                         <value>COVEREDRATIO</value>
70                         <minimum>0.80</minimum>
71                     </limit>
72                     <limit>
73                         <counter>BRANCH</counter>
74                         <value>COVEREDRATIO</value>
75                         <minimum>0.70</minimum>
76                     </limit>
77                 </limits>
78             </rule>
79         </rules>
80     </configuration>
81 </execution>
82 </executions>
83 </plugin>

```

Listing 12.3: Configuración JaCoCo Avanzada

12.3. Profiles Maven por Ambiente

```

1 <profiles>
2     <!-- Desarrollo -->
3     <profile>
4         <id>dev</id>
5         <activation>
6             <activeByDefault>true</activeByDefault>
7         </activation>
8         <properties>
9             <spring.profiles.active>dev</spring.profiles.active>
10            <log.level>DEBUG</log.level>
11        </properties>
12    </profile>
13
14    <!-- Testing -->
15    <profile>
16        <id>test</id>
17        <properties>
18            <spring.profiles.active>test</spring.profiles.active>

```

```

19         <log.level>WARN</log.level>
20     </properties>
21     <build>
22         <plugins>
23             <plugin>
24                 <groupId>org.springframework.boot</groupId>
25                 <artifactId>spring-boot-maven-plugin</artifactId>
26                 <configuration>
27                     <skip>true</skip>
28                 </configuration>
29             </plugin>
30         </plugins>
31     </build>
32 </profile>
33
34 <!-- Producción -->
35 <profile>
36     <id>prod</id>
37     <properties>
38         <spring.profiles.active>prod</spring.profiles.active>
39         <log.level>INFO</log.level>
40     </properties>
41     <build>
42         <plugins>
43             <!-- Minimizar JAR -->
44             <plugin>
45                 <groupId>org.springframework.boot</groupId>
46                 <artifactId>spring-boot-maven-plugin</artifactId>
47                 <configuration>
48                     <excludes>
49                         <exclude>
50                             <groupId>org.springframework.boot</
51 groupId>
52                             <artifactId>spring-boot-devtools</
53 artifactId>
54                             </exclude>
55                         </excludes>
56                     </configuration>
57                 </plugin>
58             </plugins>
59         </build>
60     </profile>
61
62 <!-- Análisis de seguridad -->
63 <profile>
64     <id>security</id>
65     <build>
66         <plugins>
67             <plugin>
68                 <groupId>org.owasp</groupId>
69                 <artifactId>dependency-check-maven</artifactId>
70                 <executions>
71                     <execution>
72                         <goals>
73                             <goal>check</goal>
74                         </goals>
75                     </execution>
76                 </executions>

```

```

75         </plugin>
76     </plugins>
77 </build>
78 </profile>
79 </profiles>

```

Listing 12.4: Profiles Avanzados Maven

12.4. Testing de Performance con JMeter

```

1 <plugin>
2   <groupId>com.lazerycode.jmeter</groupId>
3   <artifactId>jmeter-maven-plugin</artifactId>
4   <version>3.6.1</version>
5   <configuration>
6     <testFilesDirectory>${project.basedir}/src/test/jmeter</
testFilesDirectory>
7     <resultsDirectory>${project.build.directory}/jmeter/results</
resultsDirectory>
8     <jmeterProperties>
9       <property>
10        <name>server.rmi.ssl.disable</name>
11        <value>true</value>
12      </property>
13    </jmeterProperties>
14  </configuration>
15  <executions>
16    <execution>
17      <id>jmeter-tests</id>
18      <goals>
19        <goal>jmeter</goal>
20      </goals>
21    </execution>
22  </executions>
23 </plugin>

```

Listing 12.5: Plugin JMeter Maven

```

1 stage('Performance Tests') {
2   when {
3     anyOf {
4       branch 'main'
5       branch 'develop'
6     }
7   }
8   steps {
9     sh '''
10       mvn jmeter:jmeter \
11         -Dhost=${STAGING_HOST} \
12         -Dport=80 \
13         -Dprotocol=http \
14         -Dthreads=50 \
15         -Dramp-up=60 \
16         -Dduration=300
17     '''
18   }
19   post {

```

```

20     always {
21         performanceReport parsers: [
22             [$class: 'JMeterParser', glob: 'target/jmeter/results/*.
jtl']
23         ], relativeFailedThresholdNegative: 10.0,
24         relativeFailedThresholdPositive: 5.0,
25         relativeUnstableThresholdNegative: 5.0,
26         relativeUnstableThresholdPositive: 2.0
27     }
28 }
29 }

```

Listing 12.6: Pipeline con Performance Tests

12.5. Microservicios Java

12.5.1. Pipeline Multi-Módulo

Para proyectos con múltiples microservicios:

```

1 pipeline {
2     agent none
3
4     stages {
5         stage('Build All Modules') {
6             agent any
7             steps {
8                 sh 'mvn clean compile -pl "!integration-tests"'
9             }
10        }
11
12        stage('Test Modules') {
13            parallel {
14                stage('User Service Tests') {
15                    agent any
16                    steps {
17                        sh 'mvn test -pl user-service'
18                    }
19                    post {
20                        always {
21                            publishTestResults testResultsPattern: 'user
-service/target/surefire-reports/*.xml'
22                        }
23                    }
24                }
25
26                stage('Order Service Tests') {
27                    agent any
28                    steps {
29                        sh 'mvn test -pl order-service'
30                    }
31                    post {
32                        always {
33                            publishTestResults testResultsPattern: '
order-service/target/surefire-reports/*.xml'
34                        }
35                    }
36                }
37            }
38        }
39    }
40 }

```

```

36         }
37
38         stage('Payment Service Tests') {
39             agent any
40             steps {
41                 sh 'mvn test -pl payment-service'
42             }
43             post {
44                 always {
45                     publishTestResults testResultsPattern: '
payment-service/target/surefire-reports/*.xml'
46                 }
47             }
48         }
49     }
50 }
51
52 stage('Integration Tests') {
53     agent any
54     steps {
55         sh '''
56             # Start all services with Docker Compose
57             docker-compose up -d
58
59             # Wait for services to be ready
60             ./wait-for-services.sh
61
62             # Run integration tests
63             mvn test -pl integration-tests
64         '''
65     }
66     post {
67         always {
68             sh 'docker-compose down'
69             publishTestResults testResultsPattern: 'integration-
tests/target/surefire-reports/*.xml'
70         }
71     }
72 }
73
74 stage('Package and Deploy') {
75     parallel {
76         stage('Deploy User Service') {
77             agent any
78             when { changeset "user-service/**" }
79             steps {
80                 sh '''
81                     mvn package -pl user-service -DskipTests
82                     docker build -t user-service:${BUILD_NUMBER}
user-service/
83                     docker push ${REGISTRY}/user-service:${
BUILD_NUMBER}
84                 '''
85             }
86         }
87
88         stage('Deploy Order Service') {
89             agent any

```

```

90         when { changeset "order-service/**" }
91         steps {
92             sh '''
93                 mvn package -pl order-service -DskipTests
94                 docker build -t order-service:${BUILD_NUMBER}
95             } order-service/
96                 docker push ${REGISTRY}/order-service:${
97             BUILD_NUMBER}
98             '''
99         }
100     }
101 }
102 }
```

Listing 12.7: Pipeline Multi-Módulo

Capítulo 13

Casos de Uso Avanzados

13.1. Blue-Green Deployment

```
1 pipeline {
2   agent any
3
4   environment {
5     BLUE_ENV = 'blue'
6     GREEN_ENV = 'green'
7     CURRENT_ENV = ''
8     TARGET_ENV = ''
9   }
10
11   stages {
12     stage('Determine Environment') {
13       steps {
14         script {
15           // Determinar ambiente actual
16           def currentService = sh(
17             script: "kubectl get service production-service
18             -o jsonpath='{.spec.selector.version}'",
19             returnStdout: true
20           ).trim()
21
22           env.CURRENT_ENV = currentService ? : 'blue'
23           env.TARGET_ENV = (env.CURRENT_ENV == 'blue') ? '
24           green' : 'blue'
25
26           echo "Current environment: ${env.CURRENT_ENV}"
27           echo "Target environment: ${env.TARGET_ENV}"
28         }
29       }
30     }
31
32     stage('Deploy to Target Environment') {
33       steps {
34         script {
35           // Desplegar a ambiente objetivo
36           sh """
37             kubectl set image deployment/app-${env.
38             TARGET_ENV} \
39             app=${DOCKER_REGISTRY}/${APP_NAME}:${{
40             BUILD_NUMBER} \
```

```

37         -n production
38
39         kubectl rollout status deployment/app-${env.
TARGET_ENV} \
40         -n production --timeout=300s
41         ""
42     }
43 }
44 }
45
46 stage('Health Check Target') {
47     steps {
48         script {
49             // Health check del nuevo ambiente
50             def healthCheckUrl = "http://app-${env.TARGET_ENV}.
production.svc.cluster.local:8080/health"
51
52             sh ""
53             for i in {1..30}; do
54                 if curl -f ${healthCheckUrl}; then
55                     echo "Health check passed for ${env.
TARGET_ENV}"
56                     break
57                 fi
58                 echo "Waiting for health check... (\$i/30)"
59                 sleep 10
60             done
61             ""
62         }
63     }
64 }
65
66 stage('Switch Traffic') {
67     steps {
68         input message: "Switch traffic to ${env.TARGET_ENV}?",
ok: 'Switch'
69
70         script {
71             // Cambiar tr fico al nuevo ambiente
72             sh ""
73             kubectl patch service production-service \
74             -p '{"spec":{"selector":{"version":"${env.
TARGET_ENV}}}}' \
75             -n production
76             ""
77
78             echo "Traffic switched to ${env.TARGET_ENV}"
79         }
80     }
81 }
82
83 stage('Verify Switch') {
84     steps {
85         script {
86             // Verificar que el switch fue exitoso
87             sleep(30) // Esperar propagaci n
88
89             sh ""

```



```

90         for i in {1..10}; do
91             curl -f http://production.yourapp.com/health
92             sleep 5
93         done
94         ""
95
96         echo "Production verification successful"
97     }
98 }
99 }
100 }
101
102 post {
103     failure {
104         script {
105             // Rollback en caso de fallo
106             echo "Rolling back to ${env.CURRENT_ENV}"
107
108             sh ""
109                 kubectl patch service production-service \
110                 -p '{"spec":{"selector":{"version":"${env.
CURRENT_ENV}}}}' \
111                 -n production
112             ""
113         }
114     }
115
116     success {
117         script {
118             // Limpiar ambiente anterior despu s del xito
119             echo "Deployment successful. Old environment ${env.
CURRENT_ENV} can be cleaned up"
120         }
121     }
122 }
123 }

```

Listing 13.1: Pipeline Blue-Green Deployment

13.2. Pipeline Multi-Región

```

1 pipeline {
2     agent any
3
4     parameters {
5         choice(
6             name: 'REGIONS',
7             choices: ['us-east-1', 'us-west-2', 'eu-west-1', 'all'],
8             description: 'Select deployment regions'
9         )
10        booleanParam(
11            name: 'CANARY_DEPLOYMENT',
12            defaultValue: false,
13            description: 'Use canary deployment strategy'
14        )
15    }

```

```

16
17     environment {
18         AWS_REGIONS = 'us-east-1,us-west-2,eu-west-1'
19     }
20
21     stages {
22         stage('Deploy to Regions') {
23             steps {
24                 script {
25                     def regions = []
26
27                     if (params.REGIONS == 'all') {
28                         regions = env.AWS_REGIONS.split(',')
29                     } else {
30                         regions = [params.REGIONS]
31                     }
32
33                     def deploymentSteps = [:]
34
35                     regions.each { region ->
36                         deploymentSteps["Deploy to ${region}"] = {
37                             deployToRegion(region, params.
CANARY_DEPLOYMENT)
38                         }
39                     }
40
41                     parallel deploymentSteps
42                 }
43             }
44         }
45     }
46 }
47
48 def deployToRegion(String region, boolean canary) {
49     echo "Deploying to region: ${region}"
50
51     withAWS(region: region, credentials: 'aws-credentials') {
52         if (canary) {
53             // Canary deployment
54             sh """
55                 aws ecs update-service \
56                 --cluster ${region}-cluster \
57                 --service ${APP_NAME}-canary \
58                 --task-definition ${APP_NAME}:${BUILD_NUMBER} \
59                 --desired-count 1
60
61                 # Wait and monitor canary
62                 sleep 300
63
64                 # Check metrics and promote if successful
65                 aws cloudwatch get-metric-statistics \
66                 --namespace AWS/ECS \
67                 --metric-name CPUUtilization \
68                 --dimensions Name=ServiceName,Value=${APP_NAME}-canary \
69                 --start-time $(date -u -d '5 minutes ago' +%Y-%m-%dT%H:%
M:%S) \
70                 --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
71                 --period 300 \

```

```
72         --statistics Average
73         ""
74     } else {
75         // Full deployment
76         sh ""
77             aws ecs update-service \
78             --cluster ${region}-cluster \
79             --service ${APP_NAME} \
80             --task-definition ${APP_NAME}:${BUILD_NUMBER}
81         ""
82     }
83
84     // Health check
85     sh ""
86         aws elbv2 describe-target-health \
87         --target-group-arn $(aws elbv2 describe-target-groups \
88         --names ${APP_NAME}-${region} \
89         --query 'TargetGroups[0].TargetGroupArn' \
90         --output text)
91     ""
92 }
93 }
```

Listing 13.2: Despliegue Multi-Región

Capítulo 14

Conclusión

14.1. Recapitulación

A lo largo de este documento hemos cubierto:

1. **Fundamentos:** Conceptos básicos de CI/CD y Jenkins
2. **Instalación:** Múltiples métodos de instalación y configuración
3. **Pipelines:** Desde básicos hasta avanzados con ejemplos prácticos
4. **Plugins:** Ecosistema y herramientas esenciales
5. **Proyecto Práctico:** Implementación completa de una aplicación
6. **Configuración Avanzada:** JCasC, Shared Libraries, y optimización
7. **Monitoreo:** Métricas, alertas y observabilidad
8. **Mejores Prácticas:** Seguridad, performance y organización
9. **Troubleshooting:** Resolución de problemas comunes
10. **Casos Avanzados:** Blue-Green deployment y multi-región

14.2. Próximos Pasos

Para continuar tu aprendizaje:

Recomendaciones

- Practica con el proyecto de ejemplo incluido
- Explora plugins específicos para tu stack tecnológico
- Implementa métricas y monitoreo en tus pipelines
- Considera certificaciones Jenkins (CJE, CDE)
- Únete a la comunidad Jenkins y contribuye

14.3. Recursos Adicionales

- Documentación oficial de Jenkins
- Repositorio de plugins
- Jenkins en GitHub
- Blue Ocean - Interfaz moderna
- Jenkins X - Cloud Native CI/CD

Apéndice A

Referencia de Comandos

A.1. CLI de Jenkins

```
1 # Descargar Jenkins CLI
2 wget http://localhost:8080/jnlpJars/jenkins-cli.jar
3
4 # Ejecutar job
5 java -jar jenkins-cli.jar -s http://localhost:8080 build job-name
6
7 # Listar jobs
8 java -jar jenkins-cli.jar -s http://localhost:8080 list-jobs
9
10 # Ver informaci n de build
11 java -jar jenkins-cli.jar -s http://localhost:8080 console job-name
12
13 # Instalar plugin
14 java -jar jenkins-cli.jar -s http://localhost:8080 install-plugin plugin
   -name
15
16 # Reiniciar Jenkins de forma segura
17 java -jar jenkins-cli.jar -s http://localhost:8080 safe-restart
```

Listing A.1: Comandos Útiles de Jenkins CLI

Apéndice B

Plantillas de Configuración

B.1. Docker Compose para Desarrollo

```
1 version: '3.8'
2
3 services:
4   jenkins:
5     image: jenkins/jenkins:lts
6     container_name: jenkins-master
7     ports:
8       - "8080:8080"
9       - "50000:50000"
10    volumes:
11      - jenkins_home:/var/jenkins_home
12      - /var/run/docker.sock:/var/run/docker.sock
13    environment:
14      - JENKINS_OPTS=--httpPort=8080
15      - JAVA_OPTS=-Xmx2048m -Xms1024m
16    networks:
17      - jenkins-network
18
19   jenkins-agent:
20     image: jenkins/inbound-agent
21     container_name: jenkins-agent-1
22     environment:
23       - JENKINS_URL=http://jenkins:8080
24       - JENKINS_AGENT_NAME=agent-1
25       - JENKINS_SECRET=${AGENT_SECRET}
26     volumes:
27       - /var/run/docker.sock:/var/run/docker.sock
28     depends_on:
29       - jenkins
30     networks:
31       - jenkins-network
32
33   sonarqube:
34     image: sonarqube:community
35     container_name: sonarqube
36     ports:
37       - "9000:9000"
38     environment:
39       - SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true
40     volumes:
```

```

41     - sonarqube_data:/opt/sonarqube/data
42     - sonarqube_logs:/opt/sonarqube/logs
43     - sonarqube_extensions:/opt/sonarqube/extensions
44     networks:
45       - jenkins-network
46
47 volumes:
48   jenkins_home:
49   sonarqube_data:
50   sonarqube_logs:
51   sonarqube_extensions:
52
53 networks:
54   jenkins-network:
55     driver: bridge

```

Listing B.1: docker-compose.yml